

CyberShield Project Report

Project Information

Project Title: CyberShield

Course: Cryptography & Network Security Lab

Project Type: Interactive Cybersecurity Simulation Platform

Group Members: Khalid, Nauman, Hussain, Shabbir, Sakhawat

Technology Stack: React, Vite, Tailwind CSS, Flask, Flask-SocketIO, SQLite, Python cryptography utilities

CyberShield is a web-based cybersecurity learning platform that demonstrates how attacks happen and how cryptographic and network security controls reduce their impact. The project presents a professional command-center dashboard where a user can select an attack scenario, launch the attack, activate defenses, observe packet flow, read attacker and defender logs, compare insecure and secure results, and export a demo report.

1. Project Proposal

1.1 Introduction

Cybersecurity concepts are often taught theoretically through algorithms, diagrams, and definitions. However, many students find it difficult to understand how an attack actually affects a communication system and how cryptographic protection changes the outcome.

CyberShield was proposed to solve this learning gap. It combines attack simulation, cryptographic defense, real-time visualization, dual monitoring perspectives, and educational explanation into one interactive platform. Instead of only explaining AES, hashing, replay protection, or packet sniffing separately, the project shows these concepts inside practical attack-defense scenarios.

1.2 Problem Statement

Students studying Cryptography & Network Security may understand formulas and definitions but still struggle with operational questions such as:

- How does an attacker intercept data?
- What does plaintext exposure look like?
- Why does encryption matter if packets can still be captured?
- How does account lockout stop brute-force guessing?

- Why are nonces important in replay attack prevention?

Traditional learning methods often lack interactivity, live visualization, and side-by-side comparison between insecure and secure communication. CyberShield addresses this by letting users launch attacks, inspect the visible impact, activate defenses, and observe how the result changes.

1.3 Objectives

The main objectives of CyberShield are:

- Simulate real-world cyber attacks in a controlled educational environment.
- Demonstrate cryptographic protection using AES, RSA, SHA-256, digital signatures, password hashing, and nonce-based validation.
- Visualize the difference between plaintext and protected communication.
- Provide a professional dashboard with live alerts, attack metrics, packet activity, and dual hacker/defender consoles.
- Help students understand security concepts through scenario-based learning.
- Provide before-and-after comparison and Explain Mode for easier presentation and viva explanation.

1.4 Proposed Solution

CyberShield is implemented as a full-stack web application. The frontend provides the dashboard, login/register screens, visual network activity, scenario selection, Explain Mode, and report export. The backend provides authentication, cryptographic utilities, scenario simulation, API routes, and WebSocket-based real-time event streaming.

The system supports four attack scenarios:

Scenario	Attack Type	Main Concept Demonstrated
Bank Transaction Interception	Man-in-the-Middle	Plaintext interception vs AES-protected payload
Password Brute Force	Credential Attack	Guessing attack vs password hashing, lockout, and rate control
Packet Sniffing	Packet Capture	Readable traffic vs encrypted packet contents

Replay Attack	Session Abuse	Reused request vs nonce/session validation
---------------	---------------	---

2. Acknowledgment

We would like to thank our Cryptography & Network Security instructor for guiding us toward a project that connects theory with practical implementation. This project helped us understand that cryptographic algorithms are not only mathematical topics but also active defenses used inside real systems.

We also acknowledge the importance of official security standards, open-source documentation, and development tools that made the project possible. CyberShield was built as an academic learning platform, and every attack scenario is simulated in a controlled environment for educational use only.

3. Abstract

CyberShield is an interactive cyber attack and cryptography visualization platform developed for the Cryptography & Network Security lab. The project demonstrates common cyber attacks and their defenses through a real-time dashboard. It includes user authentication, scenario selection, attack launch control, defense activation, network visualization, hacker and defender terminals, live alerts, before-vs-after comparison, Explain Mode, and Markdown report export.

The backend is built with Flask and contains modular services for authentication, cryptographic operations, simulation generation, and real-time communication through Flask-SocketIO. The frontend is built with React and Tailwind CSS to provide a professional Security Operations Center style interface.

The project demonstrates AES-256-GCM encryption, RSA encryption, SHA-256 hashing, RSA digital signatures, PBKDF2-SHA256 password hashing, JWT authentication, nonce protection, replay detection, packet sniffing behavior, and brute-force mitigation. The final result is a complete educational platform that allows students to see how insecure communication can be exploited and how security mechanisms neutralize or reduce the impact of attacks.

4. Methodology

CyberShield was developed in a phase-wise manner to keep the system modular and understandable.

4.1 Requirement Analysis

The first step was to study the project proposal and context document. The required system needed to be more than a static website. It had to behave like a cybersecurity command center with attack simulations, cryptographic defenses, live logs, and educational explanations.

The main requirements identified were:

- Login/register before dashboard access.
- Backend health and API response structure.
- Cryptographic engine.
- Attack simulation engine.
- Real-time updates.
- Professional dashboard layout.
- Four attack scenarios.
- Before-and-after comparison.
- Explain Mode.
- Exportable demo report.

4.2 System Architecture

CyberShield follows a client-server architecture.

Layer	Responsibility
Frontend	UI, login/register, dashboard, visualization, user interaction
Backend API	Authentication, scenario routes, health routes, JSON responses
Crypto Engine	AES, RSA, SHA-256, digital signatures, password hashing support
Simulation Engine	Packet generation, scenario state, alerts, attack logs, defense logs
WebSocket Layer	Real-time simulation state and event streaming

Database	SQLite user storage for registered accounts
----------	---

4.3 Authentication and Session Handling

The project starts with a login/register page. A user must authenticate before accessing the dashboard. Passwords are hashed before storage, and a JWT token is used to maintain the authenticated session.

The registered users are stored locally in SQLite at:

`backend/app/instance/cybershield.sqlite`

This keeps the project easy to run locally while still demonstrating real user persistence.

4.4 Cryptographic Engine Development

The cryptographic engine was created to demonstrate course concepts practically:

- AES-256-GCM is used to show symmetric encryption and authenticated ciphertext.
- RSA-OAEP-SHA256 is used to demonstrate public/private key encryption.
- RSA-PSS-SHA256 digital signatures are used to show authenticity and integrity.
- SHA-256 is used for digest generation and request fingerprinting.
- PBKDF2-SHA256 is used for password hashing in authentication and brute-force demonstrations.

4.5 Simulation Engine Development

Each scenario has a dedicated simulation builder. The builders generate:

- Scenario metadata.
- Launch state.
- Defense state.
- Attack success or failure.
- Channel status.
- Metrics.
- Packets.
- Alerts.
- Hacker logs.
- Defender logs.
- Active defense states.

This design allows every scenario to behave differently while still returning a consistent structure to the frontend.

4.6 Frontend Dashboard Development

The frontend was designed as a professional command center. It includes:

- Scenario rail.
- Network visualization.
- Metrics cards.
- Live alerts.
- Active defenses.
- Hacker terminal.
- Defender console.
- Explain Mode.
- Before-vs-after comparison.
- Report export button.

The dashboard logo glow also communicates system status:

- Blue means neutral.
- Red means an attack is running.
- Green means the threat has been neutralized.

4.7 Real-Time Integration

Flask-SocketIO is used to stream simulation events to the frontend. This gives the dashboard a live feeling because logs, alerts, packets, and state updates are synchronized while the scenario runs.

4.8 Testing and Verification

The backend was verified through unit tests. The frontend was verified through linting and production build.

Final verification completed:

Verification	Result
Backend unit tests	26 tests passed
Frontend lint	Passed
Frontend production build	Passed

5. System Implementation Overview

5.1 Frontend Implementation

The frontend uses React components and reusable hooks. It handles authentication state, backend health, simulation data, real-time events, and scenario comparison.

Important frontend parts include:

- Login/register gate.
- Brand loader with project logo.
- Dashboard shell.
- Network visualizer.
- Scenario rail.
- Hacker and defender terminal streams.
- Explain Mode panel.
- Security comparison panel.
- Markdown report export utility.

5.2 Backend Implementation

The backend is built with Flask and follows a modular structure. It includes:

- Health route.
- Authentication routes.
- Simulation routes.
- Crypto routes.
- WebSocket event handlers.
- Response helpers.
- Scenario services.
- SQLite user storage.

All API responses follow a consistent JSON format:

```
{  
  "success": true,  
  "message": "Response message",  
  "data": {}  
}
```

5.3 Scenario Workflow

The normal user workflow is:

1. User opens CyberShield.
2. User registers or logs in.
3. Dashboard opens after authentication.
4. User selects a scenario.
5. User clicks Launch.
6. Attack simulation starts.
7. Dashboard shows packet movement, attack logs, alerts, and metrics.
8. User clicks Defend.
9. Defense simulation activates.
10. Dashboard shows the threat neutralized state.
11. User reviews Explain Mode and before-vs-after comparison.
12. User exports a Markdown demo report.

6. Results & Discussion

6.1 Bank Transaction Interception

This scenario demonstrates a Man-in-the-Middle attack.

Without defense, the transaction payload is visible in plaintext. The attacker can read sensitive transaction details such as account, merchant, amount, and nonce. With defense enabled, AES-256-GCM encrypts the transaction before it moves through the network. The attacker can still capture traffic, but the captured payload becomes unreadable ciphertext.

Result: The attack changes from successful data exposure to failed payload recovery.

6.2 Password Brute Force

This scenario demonstrates repeated password guessing.

Without defense, the attacker tries candidate passwords until the correct password is found. With defense enabled, failed login monitoring and account lockout stop the attack before the correct password is reached. Password hashing also demonstrates why stored passwords should not be saved as plaintext.

Result: The account changes from compromised to locked and protected.

6.3 Packet Sniffing

This scenario demonstrates traffic capture.

Without defense, the sniffer captures readable HTTP, FTP, and API-style packet data. Sensitive details such as email metadata, file information, and demo session tokens are

visible. With defense enabled, traffic is represented as encrypted transport, and the attacker sees ciphertext instead of meaningful content.

Result: Packet capture still occurs, but useful information is no longer exposed.

6.4 Replay Attack

This scenario demonstrates reuse of a valid request.

Without defense, the attacker captures a valid payment request and sends the same request again. The gateway accepts the duplicate request. With defense enabled, nonce/session validation and request fingerprinting detect the duplicate and reject the replay attempt.

Result: The duplicate request changes from accepted to rejected.

6.5 Dashboard Results

The dashboard successfully shows:

- Attack launch control.
- Defense activation control.
- Loading overlays for login, attack launch, and defense action.
- Intrusion and neutralization popups.
- Scenario-specific visualization.
- Live alerts.
- Active defenses.
- Hacker terminal logs.
- Defender console logs.
- Explain Mode.
- Before-vs-after comparison.
- Exported Markdown report.

6.6 Discussion

CyberShield achieves the main academic goal of connecting theory with practice. The project does not attack a real network; instead, it safely simulates attack behavior and defense outcomes. This is appropriate for a lab environment because it avoids ethical and legal risks while still showing the important security concepts.

The strongest educational feature is the before-vs-after comparison. It makes the security impact easy to understand because students can directly compare insecure communication with defended communication. Explain Mode supports viva preparation by

describing what the attack does, what the defense does, and why the selected cryptographic technique matters.

7. New Learning

This project helped us learn several important technical and academic concepts:

- How cryptographic algorithms are used inside practical systems.
- How AES protects packet contents from being readable.
- How RSA represents public/private key security.
- How SHA-256 supports hashing and request fingerprinting.
- How digital signatures provide authenticity and integrity.
- Why passwords should be stored as hashes instead of plaintext.
- How JWT authentication protects dashboard access.
- How attack simulation can be performed safely for education.
- How frontend and backend systems communicate through APIs.
- How WebSockets provide real-time dashboard updates.
- How UI design affects understanding in technical demonstrations.
- How before-and-after comparison improves explanation during viva and exhibition.

The project also improved understanding of modular development. Instead of building everything in one file, the system was divided into frontend components, backend routes, cryptographic services, simulation builders, and real-time event handlers.

8. Future Enhancements

CyberShield is complete for the current lab submission, but it can be extended further in the future.

Possible future enhancements include:

- PDF export in addition to Markdown report export.
- Role-based access control for admin, student, and viewer accounts.
- Real packet capture integration using PCAP files or controlled lab traffic.
- AI-based intrusion detection scoring.
- More attack scenarios such as SQL injection, phishing simulation, DNS spoofing, and ransomware behavior simulation.
- Cloud deployment for online access.
- PostgreSQL database support for production use.
- Multi-user cyber battle mode.
- More detailed charts for attack timeline and risk analytics.
- Teacher dashboard for evaluating student runs.

- Scenario editor for adding custom attacks and defenses.

9. References

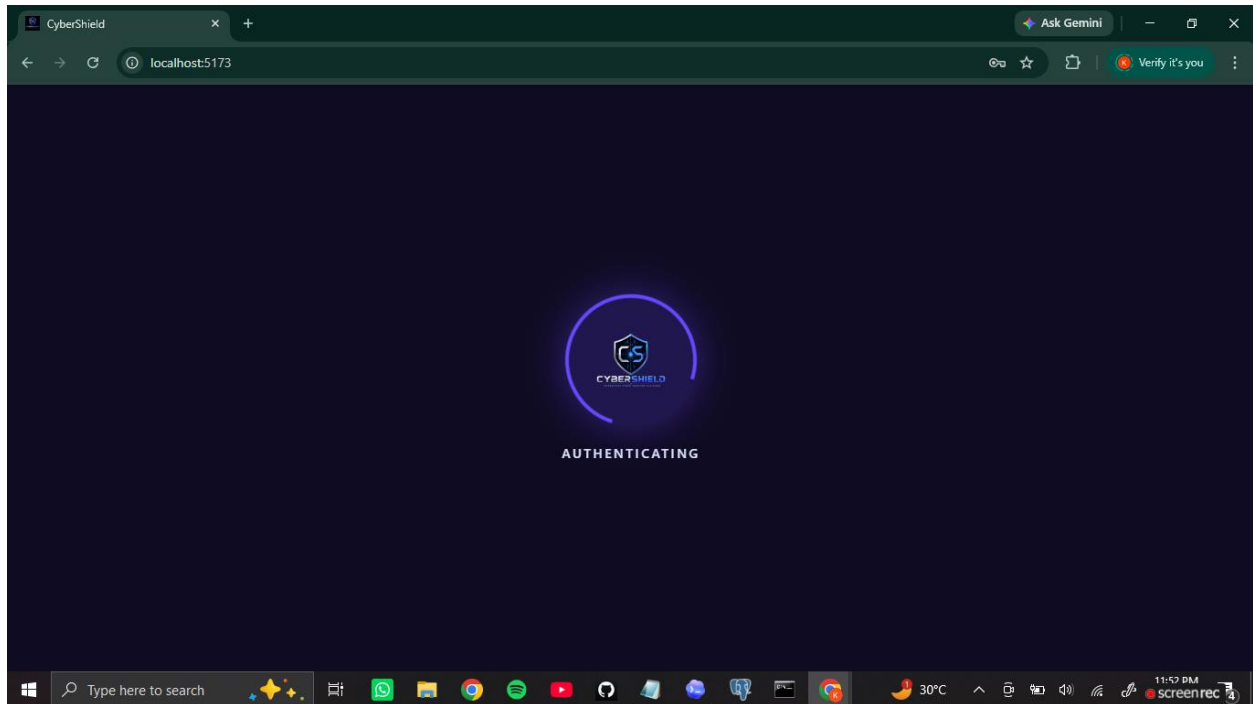
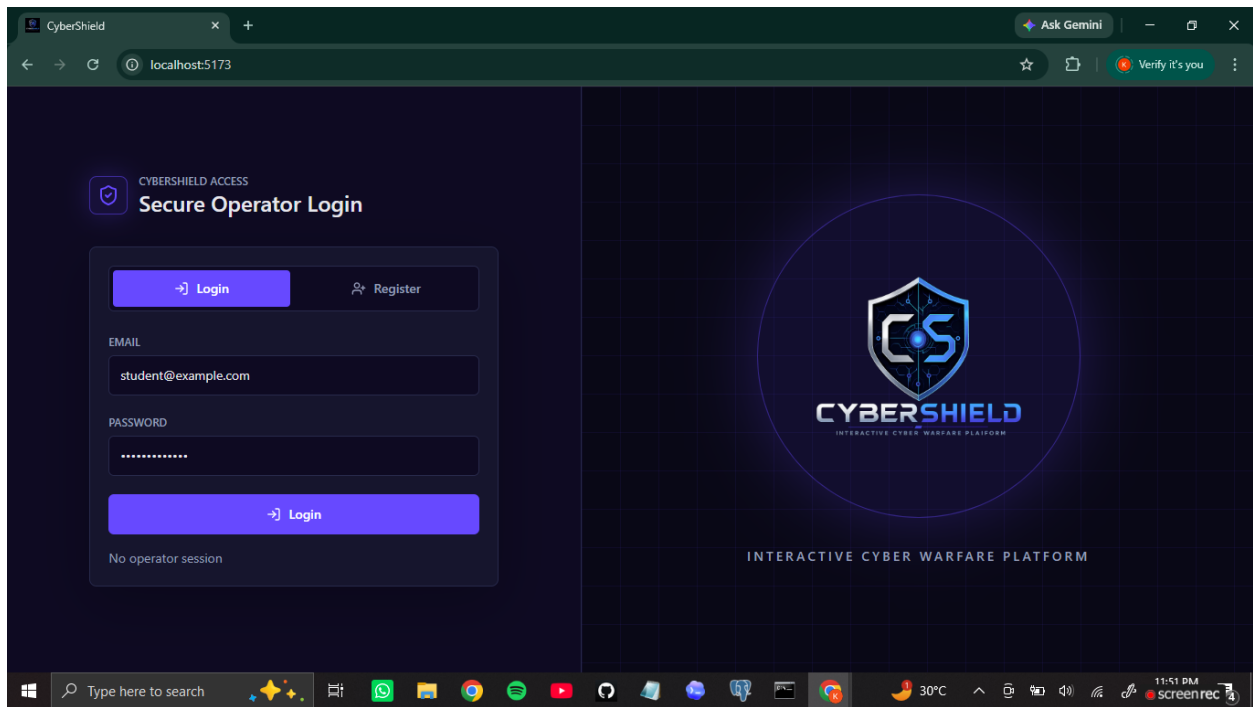
1. CyberShield Project Proposal, local project document.
2. CyberShield Project Context, local project document.
3. NIST, FIPS 197: Advanced Encryption Standard (AES),
<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf>
4. NIST, FIPS 180-4: Secure Hash Standard, <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>
5. IETF, RFC 7519: JSON Web Token (JWT),
<https://datatracker.ietf.org/doc/html/rfc7519>
6. OWASP Authentication Cheat Sheet,
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
7. Flask Documentation, <https://flask.palletsprojects.com/>
8. React Documentation, <https://react.dev/>
9. Socket.IO Documentation, <https://socket.io/docs/v4/>
10. Python cryptography Documentation, <https://cryptography.io/>

10. Conclusion

CyberShield successfully demonstrates core Cryptography & Network Security concepts through an interactive full-stack platform. It allows users to safely observe how attacks behave, how plaintext data can be exposed, and how security controls such as encryption, hashing, account lockout, and nonce validation change the outcome.

The project includes authentication, user storage, a Flask backend, a React dashboard, cryptographic services, four attack scenarios, real-time event streaming, Explain Mode, before-vs-after comparison, and demo report export. It is suitable for lab submission because it connects theoretical CNS concepts with practical demonstration in a controlled and professional environment.

Overall, CyberShield shows that cybersecurity learning becomes more effective when students can see both sides of the attack-defense process: what the attacker tries to exploit and how the defender neutralizes the threat.



CyberShield

localhost:5173

API Online v0.1.0 cybershield-api

Realtime Live Neutral Defense Ready Khalid Hussain Report Launch

Defend

CRYPTOGRAPHY & NETWORK SECURITY LAB

CyberShield Command Center

ACTIVE ATTACKS 00 scenario paused

PROTECTED CHANNELS 00 operator controlled

PACKETS OBSERVED 000 waiting for launch

ALERT PRIORITY P3 no active incident

Scenario Engine

Bank Transaction Interception MITM / Bank API Active Critical

Password Brute Force Credential Attack / Auth Server Active High

Bank Transaction Interception

Client Transaction requ... Gateway Packet relay

Attacker Intercept node

Bank API

Live Alerts

Active Defenses

AES-256 Encryption Ready

CyberShield

localhost:5173

scenario paused operator controlled waiting for launch no active incident

Scenario Engine

Bank Transaction Interception MITM / Bank API Active Critical

Password Brute Force Credential Attack / Auth Server Active High

Packet Sniffing Capture / Client Traffic Active Medium

Replay Attack Session Abuse / Payment Gateway Active High

Bank Transaction Interception

Client Transaction requ... Gateway Packet relay

Attacker Intercept node

Bank API Settlement service

Simulation Standby None standby

Live Alerts

Active Defenses

AES-256 Encryption Ready

RSA Key Exchange Ready

Digital Signatures Standby

Nonce Protection Standby

CyberShield

localhost:5173

Ask Gemini

Verify it's you

Hacker Terminal

> [standby] Bank MITM scenario is paused

Defender Console

> [standby] Defender console waiting for simulation launch

Explain Mode

Man-in-the-Middle attack with AES-256-GCM defense

Attack Purpose

The attacker positions a relay between the client and bank gateway, then tries to read the transaction payload while it is moving through the network.

Defense Purpose

AES-256-GCM encrypts the transaction before transit, so the attacker can still capture traffic but cannot understand or modify the protected payload.

Attack Flow

Defense Flow

Type here to search

30°C

11:53 PM

screenrec

CyberShield

localhost:5173

Ask Gemini

Verify it's you

Explain Mode

Man-in-the-Middle attack with AES-256-GCM defense

Attack Purpose

The attacker positions a relay between the client and bank gateway, then tries to read the transaction payload while it is moving through the network.

Defense Purpose

AES-256-GCM encrypts the transaction before transit, so the attacker can still capture traffic but cannot understand or modify the protected payload.

Attack Flow

Defense Flow

Key Takeaway

Encryption does not always stop interception, but it prevents the attacker from reading useful information.

Type here to search

30°C

11:53 PM

screenrec

CyberShield

Ask Gemini

localhost:5173

Verify it's you

Before vs After

Compares the same scenario twice: first without defenses, then with protection enabled.

WITHOUT DEFENSE

Attack Successful

Risk Score

99/100

Packets

3

Readable

3

WITH DEFENSE

Attack Blocked

Risk Score

18/100

Packets

3

Readable

0

Risk Reduced By 81 Points

The defended run changes the outcome from attack successful to attack blocked.

Without Defense

Attack Successful

CHANNEL

Plaintext Channel (exposed)

PROTECTION

None

CAPTURED PREVIEW

TXN|from=ACCT-7721|to=MERCHANT-450|amount=2450 PKR|nonce=884291

INTERCEPTED

1

TOP ALERT

Critical: Plaintext transaction exposed

With Defense

Attack Blocked

CHANNEL

Encrypted Channel (protected)

PROTECTION

AES-256-GCM

CAPTURED PREVIEW

DLFdNZAlzfQYAbu0mku7v1d3ShRAgrTauD16FTSge6KjI3z231sjavxLqqJaaNFUoxFPciDek/npL61170pE1X7H7yPzso+GvFXnLM098g==

INTERCEPTED

1

TOP ALERT

Medium: MITM relay blocked by encrypted payload

Metric Comparison

Metric	Without Defense	With Defense	Meaning
Active Attacks	01	01	MITM relay traced
Protected Channels	00	01	AES-256 active
Packets Observed	003	003	generated by simulation engine
Alert Priority	P1	P2	attack blocked

Packet Evidence

Packet	Without Defense Evidence	With Defense Evidence
pkt-001	HTTP / in_transit TXN from=ACCT-7721 to=MERCHANT-450 amount=2450 PKR nonce=884291	HTTPS / in_transit DLFdNZAlzfQYAbu0mku7v1d3ShRAgrTauD16FTSge6
pkt-002	MITM / captured TXN from=ACCT-7721 to=MERCHANT-450 amount=2450 PKR nonce=884291	MITM / blocked DLFdNZAlzfQYAbu0mku7v1d3ShRAgrTauD16FTSge6KjI3z231sjavxLqqJaaNFUoxFPciDek/npL61170pE1X7H7yPzso+GvFXnLM098g==
pkt-003	HTTP / forwarded Forwarded transaction request	HTTPS / verified Authenticated encrypted request

Defense State Changes

AES-256 Encryption

Before: Ready

Active

RSA Key Exchange

Before: Ready

Ready

Digital Signatures

Standby

Nonce Protection

Observed

CyberShield

Ask Gemini

localhost:5173

Verify it's you

Metric Comparison

Metric	Without Defense	With Defense	Meaning
Active Attacks	01	01	MITM relay traced
Protected Channels	00	01	AES-256 active
Packets Observed	003	003	generated by simulation engine
Alert Priority	P1	P2	attack blocked

Packet Evidence

Packet	Without Defense Evidence	With Defense Evidence
pkt-001	HTTP / in_transit TXN from=ACCT-7721 to=MERCHANT-450 amount=2450 PKR nonce=884291	HTTPS / in_transit DLFdNZAlzfQYAbu0mku7v1d3ShRAgrTauD16FTSge6
pkt-002	MITM / captured TXN from=ACCT-7721 to=MERCHANT-450 amount=2450 PKR nonce=884291	MITM / blocked DLFdNZAlzfQYAbu0mku7v1d3ShRAgrTauD16FTSge6KjI3z231sjavxLqqJaaNFUoxFPciDek/npL61170pE1X7H7yPzso+GvFXnLM098g==
pkt-003	HTTP / forwarded Forwarded transaction request	HTTPS / verified Authenticated encrypted request

Defense State Changes

AES-256 Encryption

Before: Ready

Active

RSA Key Exchange

Before: Ready

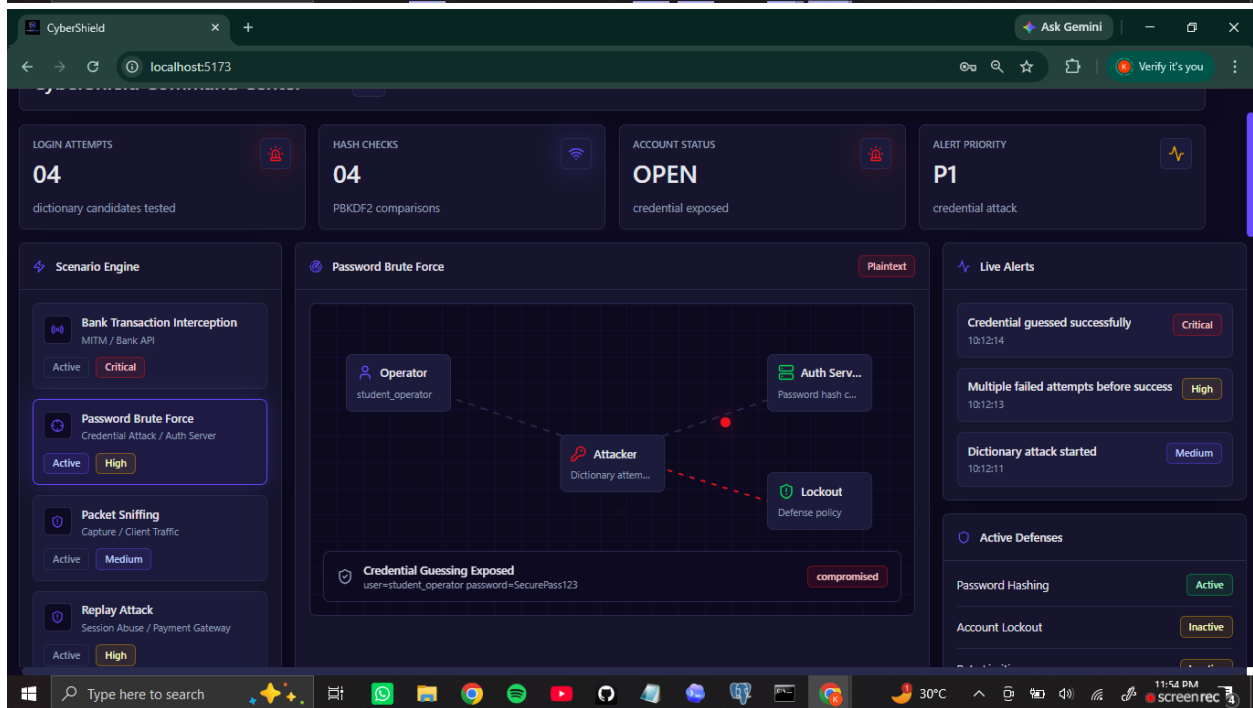
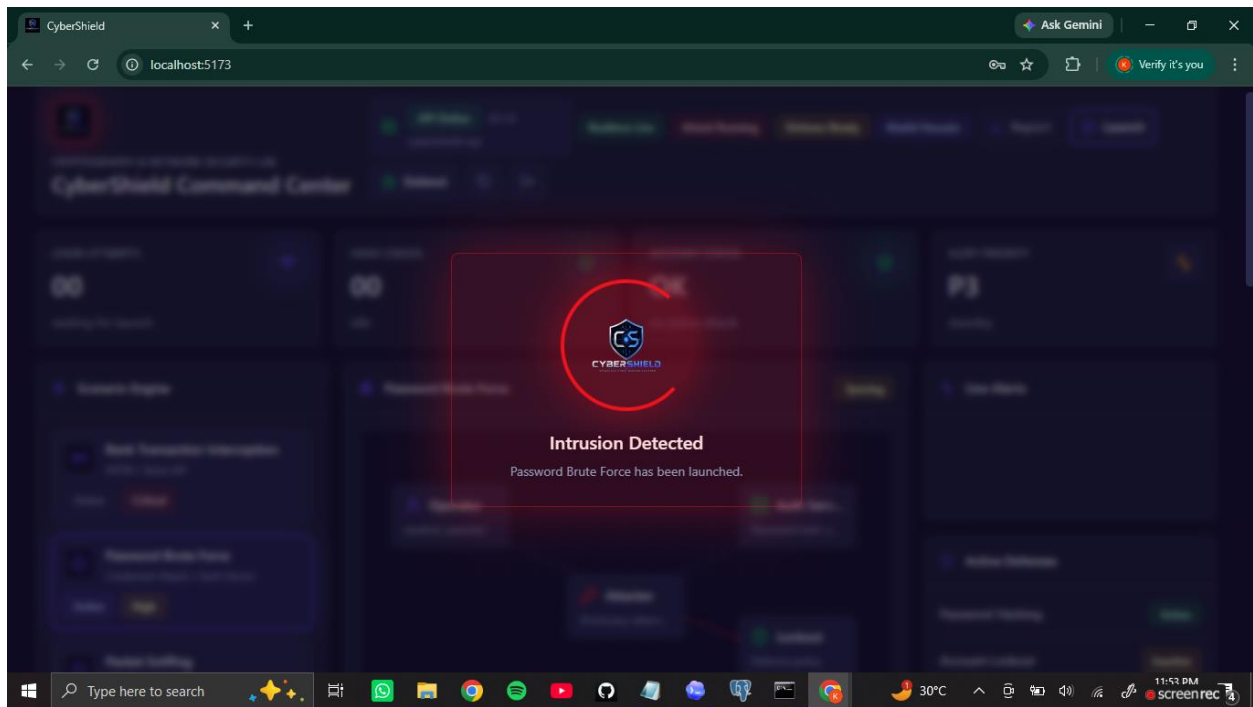
Ready

Digital Signatures

Standby

Nonce Protection

Observed



CyberShield

localhost:5173

Replay Attack
Session Abuse / Payment Gateway
Active High

Account Lockout Locked

Rate Limiting Active

Login Alerting Raised

Hacker Terminal

```
> [10:12:10] Loaded dictionary with candidate passwords
> [10:12:11] Attempt 1: password123 -> rejected
> [10:12:12] Attempt 2: admin12345 -> rejected
> [10:12:13] Attempt 3: cyberlab2026 -> rejected
> [10:12:14] Attack outcome: account locked before password discovery
```

Defender Console

```
> [10:12:10] Authentication monitor online
> [10:12:11] Failed-login counter armed
> [10:12:11] Hash check failed; counter=1
> [10:12:12] Hash check failed; counter=2
> [10:12:13] Hash check failed; counter=3
> [10:12:14] Lockout policy enforced for student_operator
> [10:12:14] High: Account lockout triggered
> [10:12:13] Medium: Repeated failed login attempts detected
> [10:12:11] Medium: Password guessing pattern observed
```

Explain Mode

Credential guessing with password hashing, lockout, and rate-control defense

Attack Purpose

Defense Purpose

11:54 PM screenrec

CyberShield

localhost:5173

Replay Attack
Session Abuse / Payment Gateway
Active High

Account Lockout Inactive

Rate Limiting Inactive

Login Alerting Raised

Hacker Terminal

```
> [10:12:10] Loaded dictionary with candidate passwords
> [10:12:11] Attempt 1: password123 -> rejected
> [10:12:12] Attempt 2: admin12345 -> rejected
> [10:12:13] Attempt 3: cyberlab2026 -> rejected
> [10:12:14] Attempt 4: SecurePass123 -> MATCH
> [10:12:14] Attack outcome: credential recovered for student_operator
```

Defender Console

```
> [10:12:10] Authentication monitor online
> [10:12:11] Lockout policy inactive
> [10:12:14] Password hash matched after 4 attempts
> [10:12:15] Alert P1 raised for compromised credential
> [10:12:14] Critical: Credential guessed successfully
> [10:12:13] High: Multiple failed attempts before success
> [10:12:11] Medium: Dictionary attack started
```

Explain Mode

Credential guessing with password hashing, lockout, and rate-control defense

Attack Purpose

Defense Purpose

11:54 PM screenrec

